EyeCon
Code Guide and Audit

How the game is built, file by file, from page load to the thesis database, plus an audit of what is solid, what is fragile and what to fix. Written so you can explain any part of the code to a panel.

Repository: EyeCon.old (master) · About 19,600 lines across HTML, JavaScript and CSS · Prepared October 2026

Contents

1. The project in one page
2. Repository map
3. How a page boots
4. The data model
5. The game loop, step by step
6. The grading engine and WCAG math
7. The editor
8. Piko and the editable text system
9. Saving: localStorage and Supabase
10. The study pipeline: quiz, consent, admin
11. The other pages and systems
12. Build, test and deploy
13. Audit findings
14. Defense cheat sheet
15. Learning path: exercises

1. The project in one page

EyeCon is a static website with no framework and no server of its own. Every page is plain HTML that loads plain JavaScript files in a fixed order. Each script wraps itself in a function, so its variables stay private, and publishes one object on `window` for the others to use. Supabase (hosted PostgreSQL) stores saves, analytics and study data, and Vercel serves the files.

Layer	What it is	Where
Pages	Website (`overview.html`), game (`index.html` , served as `/play`), test and survey (`quiz.html`), researcher dashboard (`admin.html`)	repo root
Game logic	Vanilla JavaScript modules, one global each (`window.EC_*`)	`js/`
Styling	`style.css` (base) and `reference.css` (later design pass that overrides it)	`css/`
Content	Level layouts exported from the Level Maker, then compiled into `js/level-edits.js`	`data/levels/`
Backend	Supabase: anonymous auth, 12 tables with Row Level Security, admin RPC functions	`supabase/migrations/`
Tooling	Vite (dev server), Node scripts (build, syntax check), Node test runner, Playwright	`scripts/` , `tests/`

> **The one-sentence architecture:** the browser is the whole application; it keeps the save in localStorage and mirrors it to Supabase, while Supabase only stores data and enforces who may read it.

2. Repository map

File	Lines	Global	Responsibility

`js/app.js`	2,746	(none; runs `init`)	The conductor: screens, desktop, mail app wiring, Piko tutorial flows, settings, shop, browser app, day cycle, rewards. 92 functions.
`js/editor.js`	2,291	`EC_EDITOR`	The design canvas: rendering elements, select, drag, snap, guides, zoom and pan, Design Panel, task list, hints, undo and redo.
`js/levels.js`	1,595	`EC_LEVELS`, `EC_PROJECTS`	Clients, pages, elements and goals (71 goals). Applies Level Maker layouts from `level-edits.js`.
`js/piko.js`	1,014	`EC_PIKO`	Mascot dialogue engine: script, speech bubble, spotlight, cursor demos, visual tutorial, mood videos.
`js/maker.js`	805	`EC_MAKER`	Admin-only Level Maker used to author level layouts.
`js/grading.js`	603	`EC_GRADING`	Goal checks and the score, stars and feedback for a submission.
`js/mail.js`	514	`EC_MAIL`	Inbox, client emails, compose and reply, arrivals, admin Skip and Repeat.
`js/storage.js`	363	`EC_STORE`	The save: profile defaults, load and save, workday, inbox rules, pay, XP, badges, shop.
`js/sound.js`	229	`EC_SOUND`	Sound effects and the music playlist.
`js/client-sites.js`	215	`EC_CLIENT_SITES`	The Browser app's interactive client websites.
`js/ui-helpers.js`	191	`EC_MODAL` and others	Modals, focus traps, fullscreen and small UI helpers.
`js/db.js`	158	`EC_DB`, `EC_ACTIVE`	Supabase sign-in, cloud save, analytics events, active-time clock.
`js/guidebook.js`	156	`EC_GUIDE`	The UI Survival Guide book with looping demos.
`js/cosmetics.js`	120	`EC_COSMETICS`	Shop items (wallpapers live; other categories dormant).
`js/text.js`	68	`EC_TEXT`	Registry of every player-facing string, with admin overrides from Supabase.
`js/wcag.js`	59	`WCAG`	Relative luminance and contrast ratio.
`quiz.html`	418	(page)	Consent, profile, pre-test and post-test, ISO survey, validation sheet, raffle.
`admin.html`	964	(page)	Statistics, dialogue editor, level reviews, study results and CSV, raffle.
`overview.html`	529	(page)	The public website and the test popup.
`css/reference.css`	3,338	–	Current look; overrides `style.css` (2,216).

3. How a page boots

`index.html` loads scripts in this exact order. Order matters because each file reads globals created by the files above it.

```
supabase-js (CDN) → config.js → text.js → db.js → wcag.js → sound.js → ui-helpers.js
→ level-edits.js → levels.js → cosmetics.js → storage.js → grading.js → editor.js
→ mail.js → piko.js → maker.js → client-sites.js → credits.js → app.js → guidebook.js
```

The module pattern used everywhere

```
(function(){
  const KEY = 'eyecon_profile_v1';      // private to this file
  function load(){ ... }
  function save(profile){ ... }
  window.EC_STORE = { load, save, ... }; // the only thing other files see
})();
```

This is the classic "IIFE plus namespace" pattern. It avoids a bundler, keeps names from colliding and makes the dependency order explicit in the HTML. The trade-off is that nothing checks the order for you: if `storage.js` moved above `cosmetics.js`, `defaultProfile()` would crash because `EC_COSMETICS` would not exist yet.

What happens on load

1. `db.js` starts signing in anonymously (8-second timeout). If Supabase is unreachable, the game plays offline.
2. `levels.js` builds the client and level list, then overlays Level Maker layouts from `EC_LEVEL_EDITS`.
3. `app.js` runs `init()`: loads the profile, applies settings and cosmetics, wires the mail app and editor callbacks, schedules client replies and shows the title screen.
4. When `EC_DB.ready` resolves, the cloud copy may replace the local save (whichever is newer), and published dialogue text is swapped in through `EC_TEXT.load`.

4. The data model

Element: one thing on a client's page

```
{ id:'title', type:'text' | 'rect', role:'heading' | 'nav' | 'button' | 'label' | 'decorative' |
...,
  text:'Welcome Bak!', x:160, y:96, w:640, h:72, z:2,
  fontFamily:'Quicksand', fontSize:48, fontWeight:'700', color:'#958958', bg:null,
  align:'left', radius:8, src:'assets/...png', locked:true, allow:['move'] }
```

Coordinates are canvas pixels on a fixed design canvas (for example 1920 × 1080). `locked` elements are artwork the player cannot move; `allow` limits what an editable element may change.

Level: one client page

```
{ id:'yappers-login', levelNumber:2, project:'yappers', clientName, tier:'novice',
  canvas:{ w, h, bg }, elements:[...], goals:[ goal(label, check, why, tip) ... ] }
```

Goal: one task in the task list

```
goal('Give "Welcome Bak!" the strongest contrast: at least 7:1, still gold.',
     { contrast:'AA', min:7, ids:['title'] }, why, tip)
```

The `check` object names one rule (`sameY`, `evenGapsX`, `contrast`, `minFont`, `ladder`, `stronger` and more) plus the element ids it applies to. The grading engine looks the rule up in a `switch`.

Profile: the save (localStorage key `eyecon_profile_v1`)

Field	Meaning
`completed`	Level ids the client approved.
`history`	Every reply received (stars, quote, reply text).
`workday`	`{ day, submissions, reviews, coins, xp }`: drives which emails can arrive.
`currency`, `totalXp`	Service Fees and experience.
`inventory`, `equipped`, `purchases`	Shop state.
`designs`, `revisionDrafts`	The player's edited elements per level.
`onboarding`, `revisionCount`	Which tutorial beats have played; revision counter for hint offers.
`pendingClientReply`, `readyReply`	A sent job waiting for, or holding, the client's answer.

5. The game loop, step by step

```
Title screen → Desktop → Mail (inbox from EC_STORE.commissionInbox)
  → open email → Accept → Editor opens (EC_EDITOR.open)
  → player edits → live task list (EC_GRADING.checkGoals on every change)
  → Submit → compose reply in Mail (type, attach, Send)
  → finishMissionSubmission → gradeSubmission → recordSubmission (save)
  → client reply arrives (scheduleClientReply) → approve or "Back to Editor"
  → Mark Completed → Service Fee → next email or end of day (sunset video)
```

Which emails can arrive

`commissionInbox()` in `storage.js` decides. In the study build (`TUTORIAL_ONLY`), day 1 offers levels 1 to 3 one at a time; day 2 offers level 4, then 5 and 6 together; day 3 offers level 7. A day ends when `workday.submissions` reaches `dailyTaskLimit()` (3, then 1 from day 3), and `nextWorkday()` starts the next.

Approval and pay

A job is approved when `stars ≥ 3` and every required goal is met (`missionComplete` in `app.js` and `storage.js`). Pay is the client's fixed fee (40 to 80); star and streak bonuses are switched off.

Tutorial choreography

Most of `app.js` is Piko choreography: `runPikoIntro`, `runPikoWorkspaceTour`, `initPikoComposeGuide` and others. They use three tools: `EC_PIKO.say(line, {onDone})` for a line, `pointAt(element)` for the spotlight, and body classes (`piko-focus`, `piko-allow`, `piko-lock-all`) that CSS uses to freeze everything except the control being taught. Progress is stored in `profile.onboarding` so a beat plays once.

6. The grading engine and WCAG math

Contrast (js/wcag.js)

```
channel c = value / 255
linear    = c ≤ 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ^ 2.4
L         = 0.2126 R + 0.7152 G + 0.0722 B
ratio     = (L_lighter + 0.05) / (L_darker + 0.05)
```

A pair passes when the ratio, rounded to one decimal as the player sees it, meets the requirement. The game asks for 4.5:1 on all text (`requiredRatio` ignores size); individual goals can demand more with `min:7`.

What text sits on (effectiveBg)

For text with no fill of its own, the engine finds the highest filled shape underneath the text's centre point (lower `z`, overlapping), and falls back to the canvas colour. This is how contrast is measured against cards and banners, not just the page background.

Goal checks (checkGoal)

Rule	Passes when
`sameY`, `sameX`, `sameCenterX`	All listed elements share that coordinate.
`evenGapsX` / `evenGapsY`	Gaps between neighbours are equal and inside `minGap`–`maxGap`.
`contrast`	Each element meets `min`/`max` against its effective background (and equal ratios when grouped).
`minFont`	Font size equals the target exactly (the name says "min", the rule is "exactly").
`ladder`, `bigger`	Sizes strictly decrease along the list.
`stronger`	One element's contrast beats another's.

`gradeSubmission` then turns results into a 0–100 score, stars (`scoreToStars`: 96, 85, 70, 50 thresholds), feedback and XP. Levels with `goals` are scored on goals only; the older rubric path (placement, hierarchy and others) remains for levels without goals.

7. The editor

All editor state lives in one object in `editor.js`:

```
state = { level, elements, original, selectedId, extraSel, history, future,
          tools:{ gridVisible, snap, guides, gridSize:8, gridOpacity },
          zoom, panX, panY, dragging, domNodes, onSubmit }
```

- **Rendering:** `buildCanvas` creates one DOM node per element (`createElementDom`); edits call `refreshElementDom` for that node only.
- **Snapping:** `roundTo8(n) = max(8, round(n / 8) × 8)` when snap is on.
- **Undo:** `pushHistory` stores a deep copy (`JSON.parse(JSON.stringify)`) before each change; `undo` and `redo` move between two stacks.
- **Zoom and pan:** a CSS transform on the canvas, eased toward a target zoom (`ZOOM_EASE`), with wheel zoom anchored at the cursor (`zoomAt`).
- **Live tasks:** `updateLiveScore` re-runs `checkGoals` after every change and ticks tasks; from level 4 the ticks and counter are hidden by design.
- **Read-only rendering:** `EC_EDITOR.renderStatic` draws a design without editing, used by mail attachments and the Browser app.

8. Piko and the editable text system

EC_TEXT (js/text.js)

Every player-facing string is registered with a key (for example `piko.handoff.0`) and a default. On start, published overrides from the `content_text` table replace defaults, so researchers can edit dialogue in `admin.html` without a deploy. `t(key, vars)` fills `{name}`-style placeholders and remembers which key produced the text, which is how `keyFor(text)` can identify a filled-in line later.

EC_PIKO (js/piko.js)

- `SCRIPT` holds the dialogue tree; it is bound to `EC_TEXT` under the `piko.` prefix.
- `say(line, opts)` queues admin-added "before" and "after" lines, then `sayOne` types the line into the bubble.
- The visual tutorial ("Show me") turns action lines into short cues with an animated hand.
- **Moods:** `MOODS` maps keys (or exact text) to a clip (`happy`, `excited`, `thinking`, `sad`). Two `<video>` players alternate so a clip swaps in only once it is playing (no blank frame). Idle is mostly `idle-blink` with a 1-in-5 `idle-look`. Safari keeps the still picture because it cannot show transparent WebM.

9. Saving: localStorage and Supabase

The game is local-first. `EC_STORE.save` writes localStorage immediately and asks `EC_DB.queueSave` to upload; uploads are debounced by 1.2 seconds and flushed on `pagehide`. On start, whichever copy is newer wins (cloud if its `updated_at` is at least the local `savedAt` minus 2 seconds).

Table	Written by	Holds
`profiles`	db.js	Whole save as JSON (`state`), name, coins, XP.
`submissions`	db.js `logSubmission`	One row per client reply: stars, score, approved, design.
`level_progress`	db.js	Best stars and completion per player and level.
`play_events`	db.js `track`	Analytics: level_open, submit, reply, hint, level_leave, with attempt and active time.
`respondents`, `quiz_results`, `raffle_entries`	quiz.html	Study profile, test answers and survey, raffle.
`content_text`, `levels`, `clients`, `level_versions`	admin.html	Editable text and level content.
`admins`	(manual)	Who counts as an admin (read only by `is_admin()`).

Security model: the browser holds only the public key. Row Level Security limits players to their own rows (`player_id = auth.uid()`); researchers read everything through `is_admin()`. Admin actions are `SECURITY DEFINER` functions that check `is_admin()` first.

Active time: `EC_ACTIVE` counts time only while the tab is visible and the player has touched the mouse or keyboard within the last 60 seconds. That is the "completion time" in the gameplay logs.

10. The study pipeline

```
overview.html popup → quiz.html?mode=pre
  consent (I Agree / I Do Not Agree) → profile → 10 items → quiz_results(kind='pre')
play the game (index.html)
quiz.html?mode=post → same 10 items → ISO/IEC 25010 survey (+ validation sheet for professionals)
  → quiz_results(kind='post', answers include { eval:{FS1..PE4, IV1..IV7}, comments })
admin.html → Study tab: pre vs post, items, survey means, respondent CSV
```

- **Item bank:** `ALL_Q` holds 20 items; `HIDDEN` hides 10, leaving the 10-item test. Answers keep each item's original number `q`, so results stay comparable if the visible set changes.
- **Pairing:** pre-test and post-test are matched by the anonymous Supabase user id, so both must be taken in the same browser.
- **Scoring:** computed in the browser and inserted directly. The admin page re-scores everyone on the current visible items.
- **Professionals:** `isPro()` checks the respondent type and adds the Instrument Validation Sheet to the post-test.

11. The other pages and systems

- **overview.html:** the public site at `/`; opens the test in an iframe popup and talks to it with `postMessage` (close, done, play).
- **client-sites.js:** turns a level's elements into an interactive website for the Browser app (nav subpages, a moving highlight pill, stretched to fill any window with `fill()`).
- **guidebook.js:** the UI Survival Guide spreads; each rule has a CSS-animated demo sharing one clock so page turns don't restart them.
- **cosmetics.js and the Shop:** five wallpapers are live; window themes and other categories are defined but hidden (`CATEGORY_ORDER = ['wallpaper']`).
- **sound.js:** sound effects and a music playlist (music1 → music2 → music3, looping), started after the first user interaction because browsers block autoplay with sound.
- **maker.js:** the Level Maker. Its exports go to `data/levels/*.json`, and `node scripts/build-level-edits.mjs` compiles them into `js/level-edits.js`.

12. Build, test and deploy

Command	Does
`npm run dev`	Vite dev server (port 4173 in this project's workflow).
`npm test`	`check:syntax` (`node --check` on 14 scripts) then the unit tests (wcag, grid, grading, levels, store).

`npm run test:e2e`	Playwright: about 35 browser tests of the app and client sites.
`npm run build`	Copies the site into `dist/`: `overview.html` becomes `index.html`, the game becomes `play.html`, and game CSS and JS URLs get `?v=<commit>` for cache busting.

Vercel runs `npm run build` and serves `dist/` with security headers (`nosniff`, `SAMEORIGIN` frames, a strict referrer policy) and one-year immutable caching for `/css` and `/js`.

13. Audit findings

Severity is about risk to the thesis and to users, not code style. "High" means it can produce wrong study data, a security problem or a broken deploy.

#	Severity	Finding	Why it matters	Fix
1	**High**	The large-text test item's key is 3:1 (WCAG AA), while the guide and lessons teach 7:1 for headings and say large text needs 7:1.	The game teaches against its own test item; Contrast barely improved. Panels will notice.	Keep the key; fix the guide text (3:1 minimum, 7:1 as EyeCon's AAA-based target) and explain the gap in the paper.
2	**High**	`quiz.html`, `admin.html` and `overview.html` load `/js` and `/css` without the `?v=` version; only `play.html` gets it, but all of `/js` is cached as immutable for a year.	After a deploy, returning visitors can run old `db.js`, `levels.js` or `config.js` on those pages.	Apply the same version rewrite to every HTML file in `build-static.mjs`.
3	**High**	Test scores, survey answers and game progress are computed in the browser and inserted directly.	A technical user could submit any score for themselves (they cannot touch others' rows).	Acknowledge as a limitation; optionally re-score from `answers` server-side (the admin page already re-scores).
4	**Medium**	The CI workflow runs on pushes to `main`, but the repository's branch is `master`.	Tests never run automatically on push.	Change the branch filter to `master`.
5	**Medium**	`check-syntax.mjs` lists 14 files and skips `text.js`, `db.js`, `guidebook.js`, `credits.js`, `dialogue-flow.js`, `quiz-art.js`, `config.js` and all inline page scripts.	A syntax error in those files ships silently.	Glob `js/*.js` instead of a hand list.
6	**Medium**	Supabase JS loads from a CDN by major version (`@2`) without integrity hashes.	Any new 2.x release reaches players untested; a compromised CDN could run code.	Pin an exact version and add `integrity`, or vendor the file.
7	**Medium**	Admin functions are executable by the `anon` role (they check `is_admin()` inside, so no data leaks).	Defense in depth; Supabase's advisor flags it.	`REVOKE EXECUTE ... FROM anon`; fix `touch_updated_at`'s search path.

8	**Medium**	The paper's Chapter 4 shows the luminance threshold as 0.04045; the code uses 0.03928.	Both appear in W3C materials and give the same results for 8-bit colours, but the paper should match the code.	Use 0.03928 in the paper, or note both.
9	**Low**	`app.js` is 2,746 lines with 92 functions, mixing tutorial scripting, UI and game rules.	Hard to test and change safely.	Split Piko flows, shop and settings into their own files.
10	**Low**	Dormant code: 15 clients defined, 7 used; the legacy rubric path in `grading.js`; hidden shop categories; `isLargeText` unused; leftover lines in `gridColumnPositions`.	Readers (and panelists) may think it is live.	Delete or mark clearly as future content.
11	**Low**	`minFont` means "exactly this size".	Misleading name for future maintainers.	Rename to `exactFont`.
12	**Low**	Admin Skip and Repeat buttons ship to production, hidden by CSS unless `?admin`.	Anyone adding `?admin` can skip levels in their own save (no study impact on others).	Fine for the study; remove before a public release.
13	**Low**	`level_leave` events are rarely recorded (29 of about 2,200 events).	Drop-off analysis is unreliable.	Also send it on `pagehide` with `sendBeacon`.
14	**Low**	Several tutorial waits poll with `setInterval` every 200 ms.	Works, but costs CPU and can leak if a step never finishes.	Use events or a `MutationObserver` (already used in the compose guide).

What is solid

- Row Level Security on every table, with admin actions behind `is_admin()`; only the public key is in the browser.
- Local-first saving: the game keeps working offline and syncs later.
- Pure, testable core: `wcag.js` and `grading.js` have no DOM dependencies and have unit tests.
- Content separated from code: levels come from Level Maker JSON, dialogue from `EC_TEXT` with admin overrides.
- Accessibility care in the UI itself: reduced-motion support, AA-checked badge colours, focus handling in modals.

14. Defense cheat sheet

Likely question	Answer
Why no framework?	The game is a canvas of positioned elements plus modals; vanilla JavaScript keeps it light, needs no build step to run, and every file is readable on its own. Vite is only the dev server.
How does the game know contrast is right?	It computes WCAG relative luminance for the text colour and for the colour actually behind it (`effectiveBg`), then the ratio (L1 + 0.05) / (L2 + 0.05).
How is a level graded?	Each task is a rule object; `checkGoal` evaluates it on the current elements. Approval needs every required task met and at least 3 stars.
Where is data stored and is it safe?	localStorage first, mirrored to Supabase. Each player signs in anonymously and RLS lets them touch only their own rows. Researchers read through admin-only functions.
Can a player cheat?	On their own data, yes: scores are computed in the browser. They cannot change anyone else's data. This is listed as a limitation.
How do you match pre-test and post-test?	By the anonymous Supabase user id of the browser, which is why both tests are taken on the same device and browser.
How was completion time measured?	`EC_ACTIVE` counts only visible-tab time with input in the last 60 seconds, so idle time is excluded.
How can researchers change dialogue?	Every string has a key in `EC_TEXT` ; the admin page saves drafts to `content_text` and publishing makes them live on the next load.
How are levels authored?	In the Level Maker; its JSON export is compiled into `level-edits.js` and applied over the level definitions.
How do you deploy?	Push to GitHub; Vercel runs the build script and serves `dist/` with cache-busted game assets and security headers.

15. Learning path: exercises

Do these in order with the dev server running. Each one makes you touch one system end to end.

1. **Read the math:** in the browser console, run `WCAG.contrastRatio('#958958', '#ffffff')` and confirm 3.5. Then work it out by hand with the formula in Section 6.
2. **Trace a goal:** find the Yappers "Welcome Bak!" goal in `levels.js` , then follow `contrast` through `checkGoal` in `grading.js` . Change `min:7` to `min:6` and watch the task tick earlier.
3. **Inspect the save:** run `EC_STORE.load()` , look at `workday` and `completed` , then skip a level with `?admin` and look again.
4. **Follow a submission:** put a breakpoint in `finishMissionSubmission` (app.js), submit a level, and step into `gradeSubmission` and `recordSubmission` .
5. **Watch the network:** open DevTools → Network, filter "supabase", play a level and match each request to a table in Section 9.

6. **Edit dialogue live:** change a Piko line in `admin.html`, publish, reload the game, and find where `EC_TEXT.load` applied it.
7. **Add a Piko mood:** add a key to `MOODS` in `piko.js` and trigger the line with `EC_PIKO.say(...)`.
8. **Write a test:** add a case to `tests/unit/wcag.test.mjs` for black on white (21:1) and run `npm test`.
9. **Fix an audit item:** start with #5 (glob all scripts in the syntax check). It is small, safe and teaches the build scripts.

Line counts and behaviour were read from the repository on the date above. If code changes, re-check the specific function before quoting it in the defense.